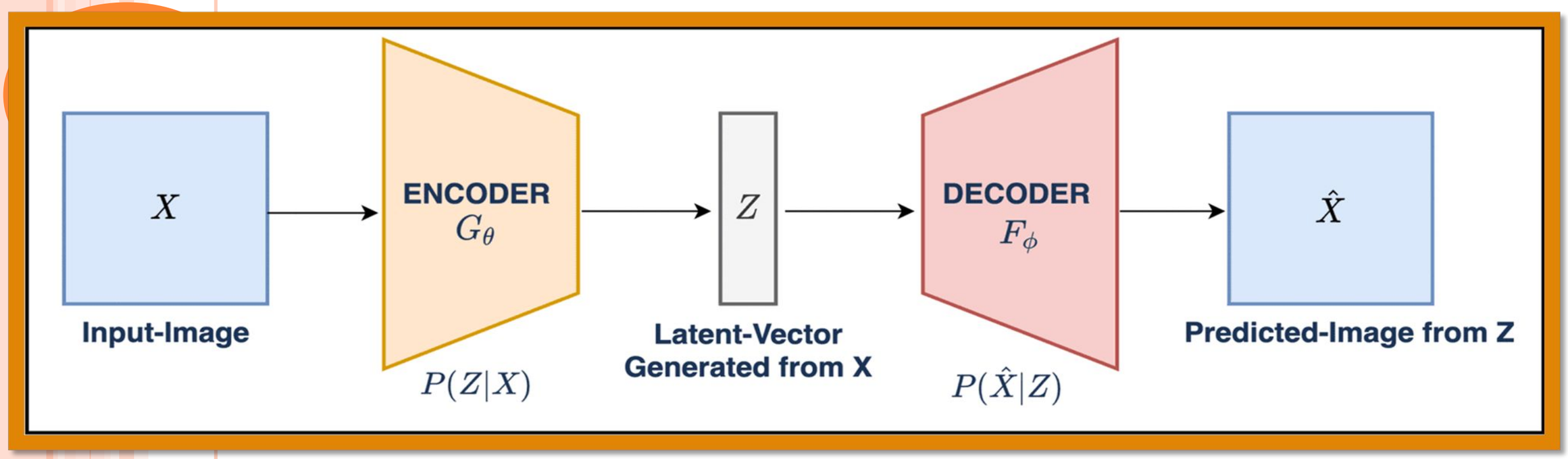
UNIT-II

Variational Auto Encoders



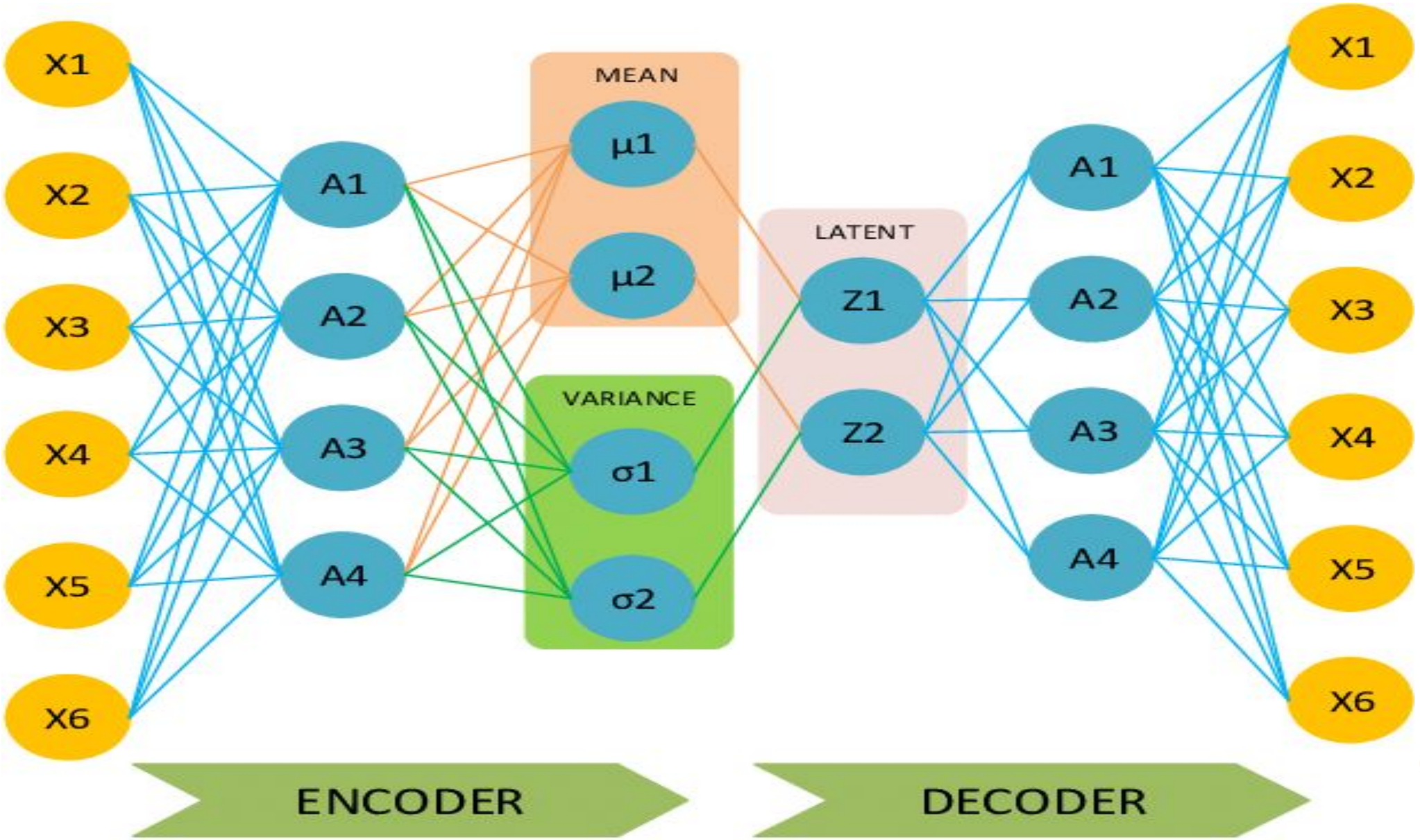
INTRODUCTION

- A more **recent type of autoencoder model** is the **Variational autoencoder (VAE)** introduced by ***Kingma and Welling*** at Google introduced in the **year 2013.**
- A **Variational Autoencoder (VAE)** is a **generative deep-learning model** that **learns the probability distribution of input data** in a **latent space.**
- Unlike a **normal autoencoder**, which **converts an input into a fixed latent vector**, a **VAE represents each latent feature using a probability distribution,** usually defined by:

  μ=Mean σ=Standard Deviation

- A **latent vector 'z'** is **sampled from this distribution** and **given to the decoder.**
- **Main purpose:** A VAE can **reconstruct existing data and generate new data similar to the training data.**

ARCHITECTURE



Components:

1. Encoder (Inference Network):

- The **encoder network takes raw input data and transforms it into a probability distribution within the latent space**.
- Maps **input x to a distribution** over **latent variables z:**

$$q_\phi(z|x) \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

- Outputs **mean** (μ) and **log-variance** (logσ2).
- i.e, the **latent code generated by the encoder** is a **probabilistic encoding,** allowing the **VAE to express not just a single point in the latent space** but a **distribution of potential representations**.

2. Latent Space:

- A **latent vector (z)** is sampled from this **learned distribution** using the **reparameterization trick.**
- The encoder **doesn't produce a fixed z**, but a **distribution**:

- Sample $z \sim \mathcal{N}(\mu, \sigma^2)$
- Use reparameterization trick:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0,1)$$

- **Sampling means picking random values** from a **probability distribution.**

For example:

- From a **coin flip distribution** (Bernoulli), sampling gives **either 0 (tails) or 1 (heads).**
- From a **normal distribution N(0,1), sampling might give values** like **0.1, -0.5, 1.3, etc.**

3. Decoder(Generative Network):

- The **decoder** takes the **sampled latent vector (z)** and **reconstructs the original data.**

In reconstruction Phase, instead of directly sampling from **z_mean & Z_log_var** , we use re-parameterized trick by the use of **Back propagation Algorithm.**

In our example, you **approximate z using the decoder parameters** and **another parameter ε** as follows:

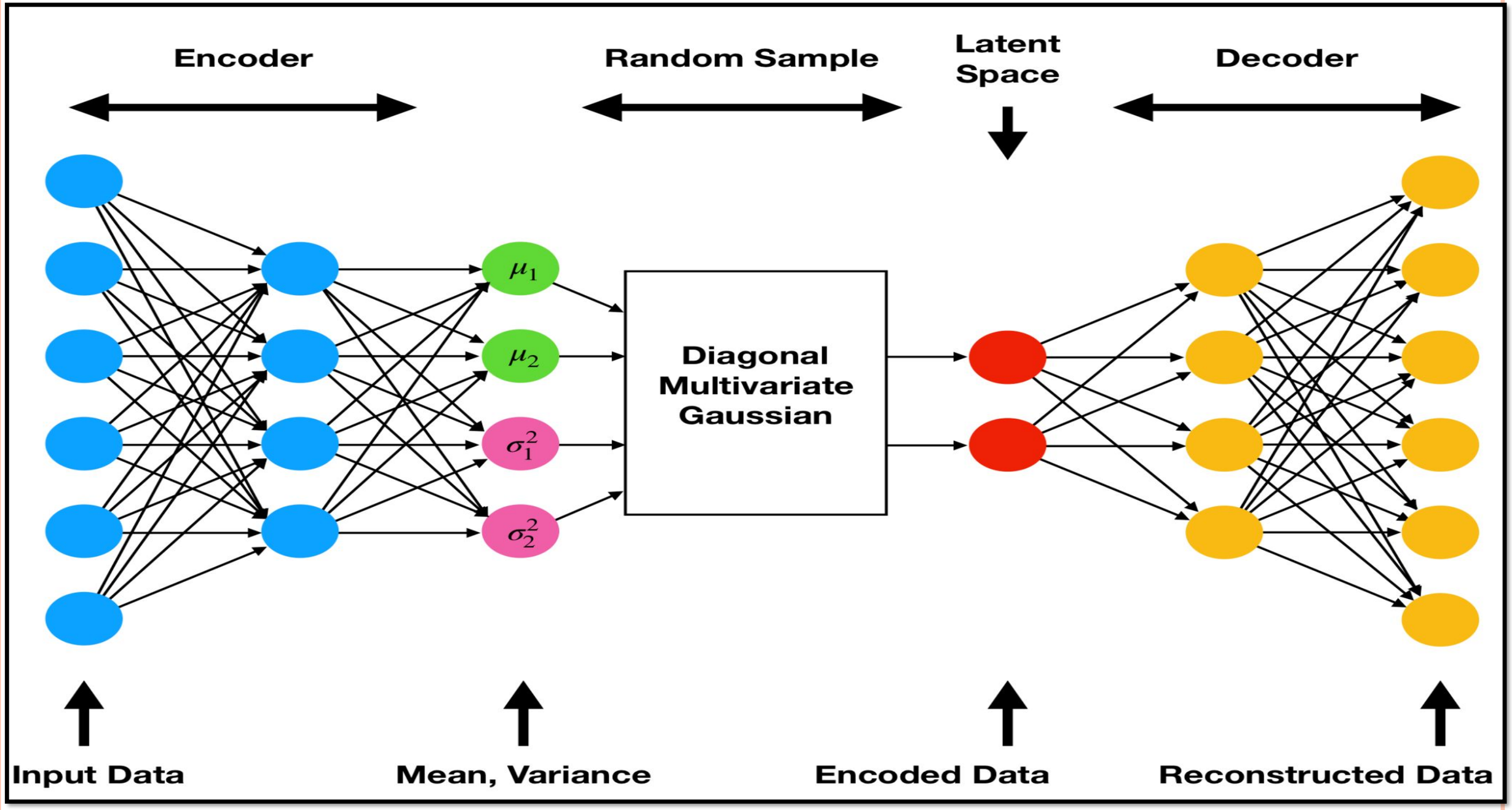
$$z_i = \mu_i + \sigma_i . \epsilon$$
$$\epsilon \sim \mathcal{N}(0,1)$$

We start by **Sampling epsilon** from a standard normal distribution.

where **μ and σ represent** the **mean and Variance of a Gaussian distribution** respectively. They can be **derived from the decoder output.**

Here, we can add the Epsilon random value between (0,1).

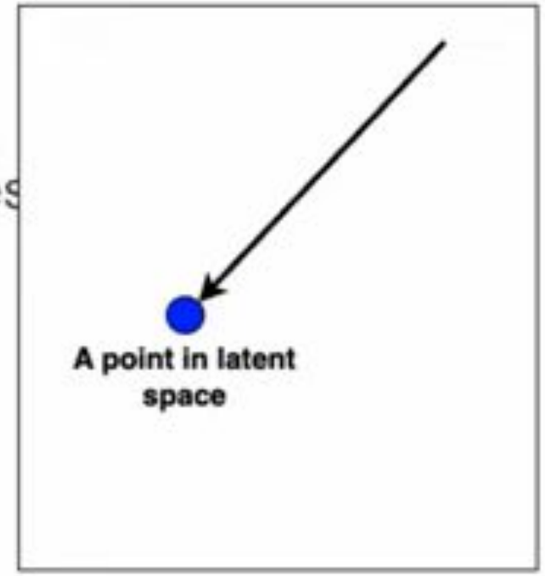



Encoder
Random Sample
Latent Space
Decoder
μ_1
μ_2
σ_1^2
σ_2^2
Diagonal Multivariate Gaussian
Input Data
Mean, Variance
Encoded Data
Reconstructed Data

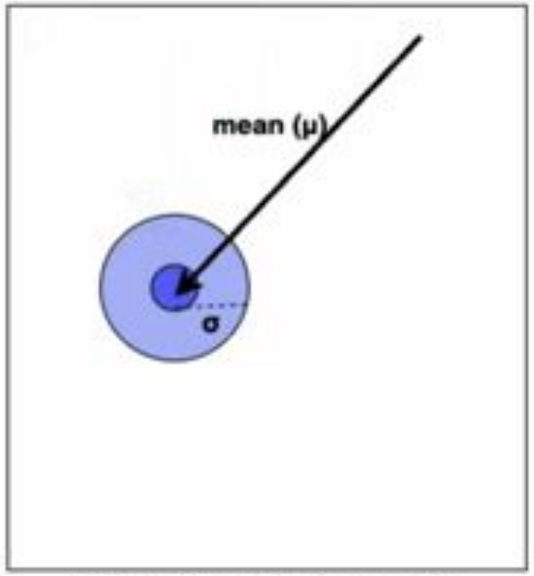
What is the use of two terms

μ and σ?

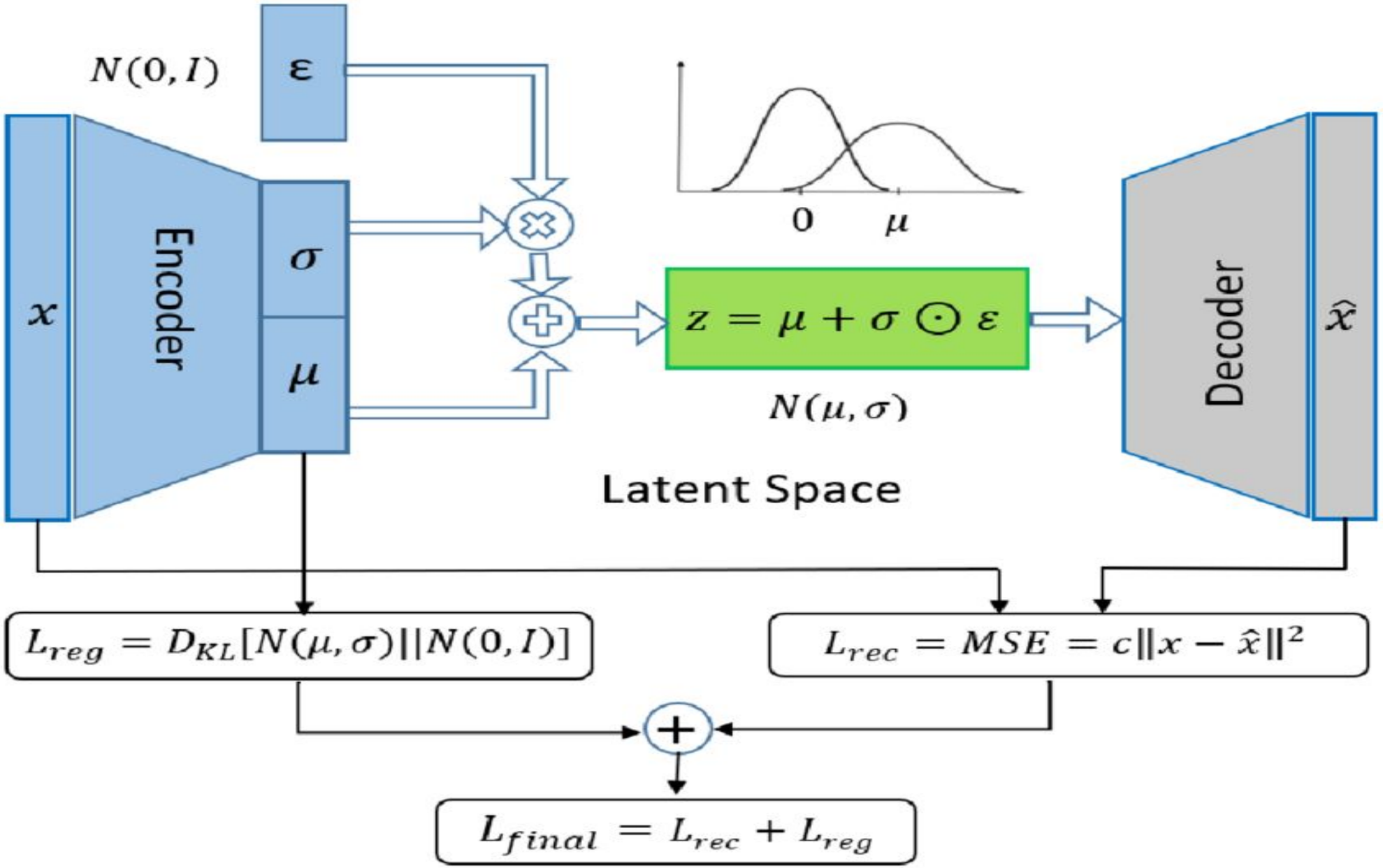
- **Mean vector (μ)** determines the central point for encoding an input
- **Standard deviation (σ)** determines how much the encoding can deviate from that central point



Vanilla Autoencoder



Variational Autoencoder
(μ and σ initialise a probability distribution)


$N(0, I)$
ε
x
Encoder
σ
μ
0
μ
$z = \mu + \sigma \odot \varepsilon$
$N(\mu, \sigma)$
Latent Space
Decoder
$\hat{x}$
$L_{reg} = D_{KL}[N(\mu, \sigma)||N(0, I)]$
$L_{rec} = MSE = c\|x - \hat{x}\|^2$
$L_{final} = L_{rec} + L_{reg}$

VARIATIONAL AUTOENCODER (VAE) – STEP BY STEP EXAMPLE

Example: Face Image → Two Dimensional Latent Space

1. INPUT (x)

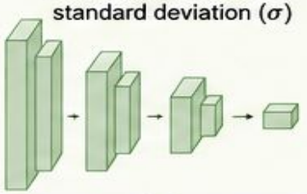
Input face image
(64 × 64 × 1)



x = original face

2. ENCODER

Encoder (Neural Network) extracts features and produces mean (μ) and standard deviation (σ)



Mean (μ)

$$\mu = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}$$

Std. Dev (σ)

$$\sigma = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$

3. RANDOM NOISE (ϵ)

Sample random noise from $N(0, I)$

$$\epsilon \sim N(0, I)$$

$$\epsilon = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}$$

(I is the identity matrix)

4. CALCULATE LATENT VECTOR (z)

Use reparameterization trick: $z = \mu + \sigma \odot \epsilon$

$$\sigma \odot \epsilon = \begin{bmatrix} 0.2 & 0 \\ 0 & 0.4 \end{bmatrix} \odot \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}$$

$$\sigma \odot \epsilon = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

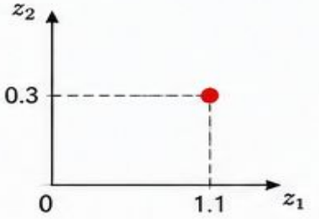
$$z = \mu + \sigma \odot \epsilon$$

$$= \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$z = \begin{bmatrix} 1.1 \\ 0.3 \end{bmatrix}$$

5. LATENT VECTOR (z)

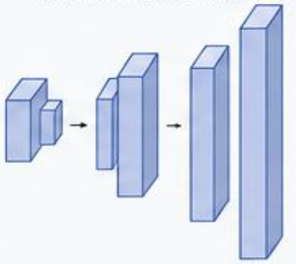
2D latent vector (point in latent space)



$$z = \begin{bmatrix} 1.1 \\ 0.3 \end{bmatrix}$$

6. DECODER

Decoder (Neural Network) reconstructs the face from latent vector z



7. RECONSTRUCTED OUTPUT ($\hat{x}$)

Reconstructed face



$\hat{x}$ = reconstructed face

8. RECONSTRUCTION LOSS (L_{rec})

Compare original and reconstructed face



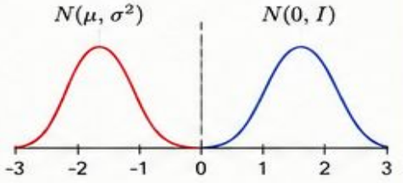
Original (x) — Reconstructed ($\hat{x}$)

$$L_{rec} = MSE = ||x - \hat{x}||^2$$

Example: $L_{rec} = 0.12$

9. KL DIVERGENCE LOSS (L_{KL})

Make the learned distribution close to prior $N(0, I)$



$$L_{KL} = D_{KL}\left[N(\mu, \sigma^2) \;||\; N(0, I)\right]$$

Example: $L_{KL} = 0.03$

10. TOTAL LOSS & TRAINING

Total loss

$$L_{final} = L_{rec} + L_{KL}$$

Example: $L_{final} = 0.12 + 0.03 = 0.15$

- Backpropagate the loss.
- Update weights of encoder and decoder.
- Repeat for many training images until loss becomes small.

SUMMARY OF FLOW

Input (x)
↓
Encoder
↓
μ, σ
↓
Sample $\epsilon \sim N(0, I)$
↓
$z = \mu + \sigma \odot \epsilon$
↓
Decoder
↓
Reconstructed ($\hat{x}$)
↓
Loss ($L_{rec} + L_{KL}$)
↓
Update & Repeat

GENERATING NEW FACES AFTER TRAINING (Sampling from Latent Space)

Sample z from $N(0, I)$
(No input image needed)
Example:

$$z = \begin{bmatrix} 0.2 \\ -0.7 \end{bmatrix}$$

→ Decoder (Trained) →



Generated Face 1

Example:

$$z = \begin{bmatrix} -1.0 \\ 0.5 \end{bmatrix}$$

→ Decoder (Trained) →



Generated Face 2

Example:

$$z = \begin{bmatrix} 0.7 \\ 1.2 \end{bmatrix}$$

→ Decoder (Trained) →



Generated Face 3

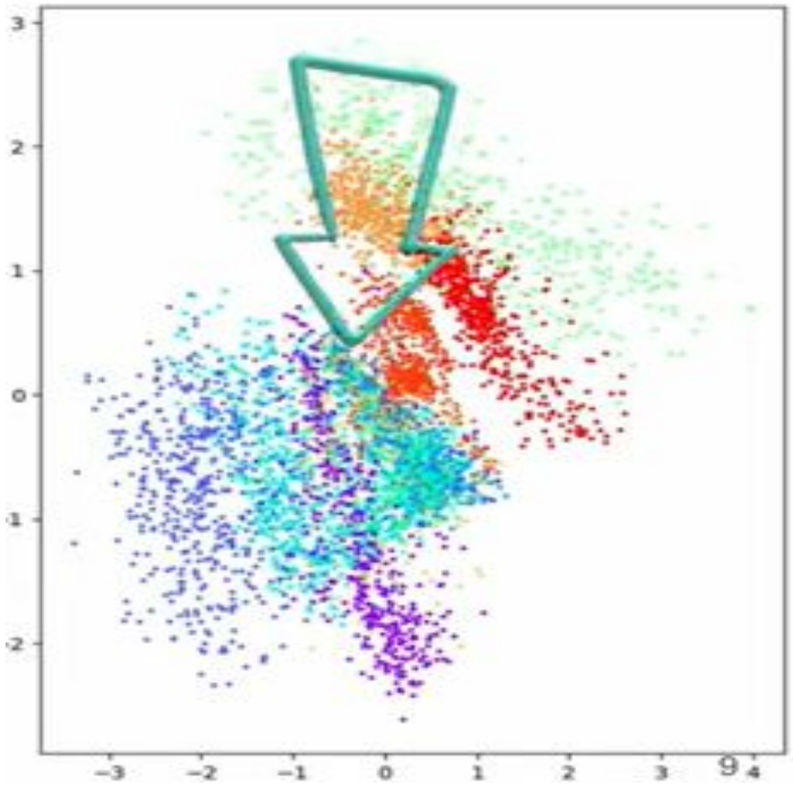
Different z → Different New Faces

NOTES: • $\odot$: Element-wise multiplication • I : Identity matrix • MSE : Mean Squared Error • D_{KL} : Kullback–Leibler Divergence

Why we need both Reconstruction and KL Loss

When using only the **KL Loss** , the **Latent Space ends up with encodings scattered randomly near its centers**, without considering similarity among nearby encodings.

When optimizing **both the KL Loss and another Loss together**, it creates a latent space where nearby encodings are similar, forming clusters on a local scale.



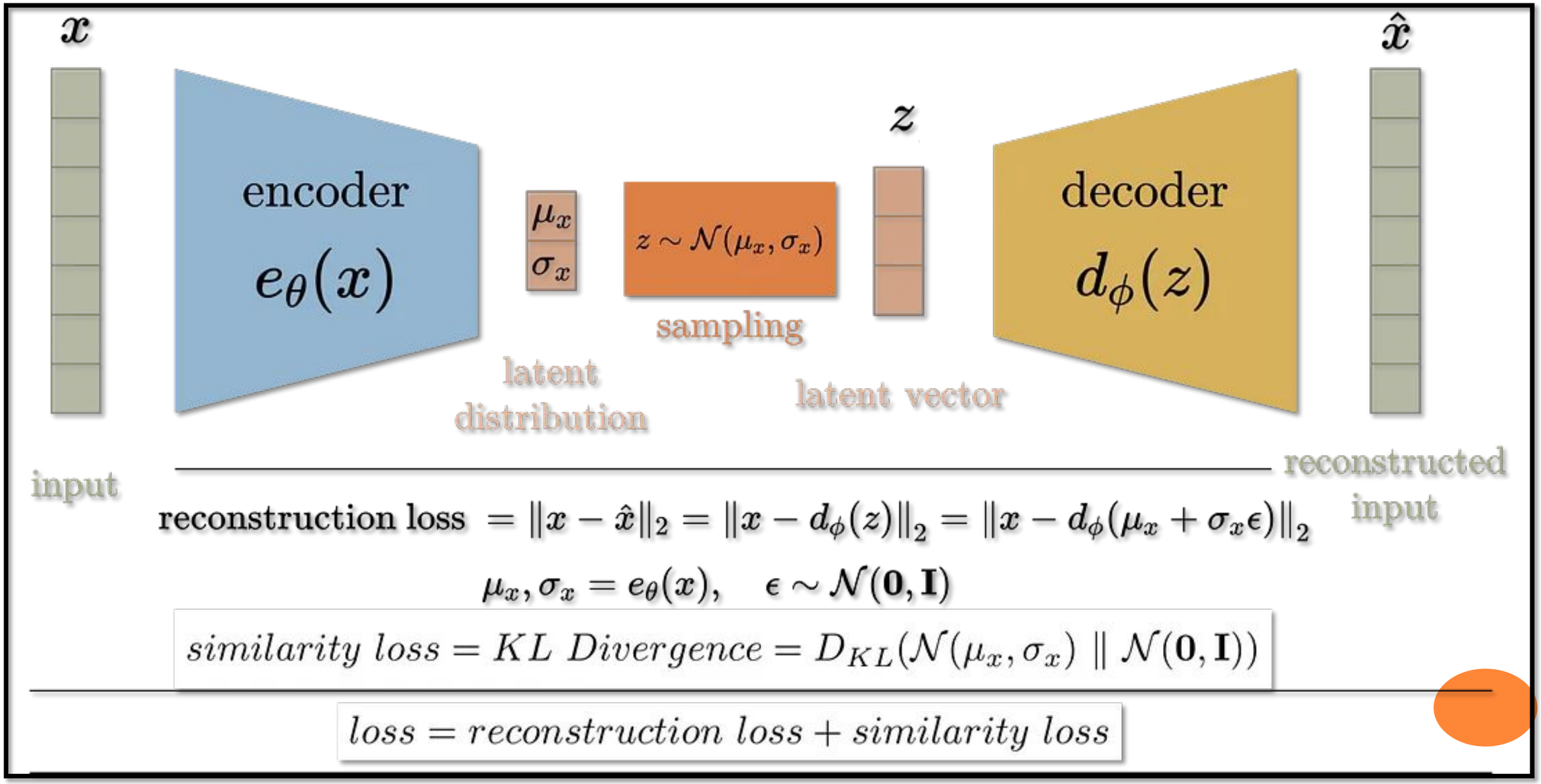
- **Reconstruction Loss:** Ensures that the reconstructed data is similar to the original input.
- **KL Divergence Loss:** Encourages the learned distribution to be close to a standard normal distribution , **preventing overfitting.**

🔷 VAE Loss = Reconstruction Loss + Latent Regularization

This can be written as:

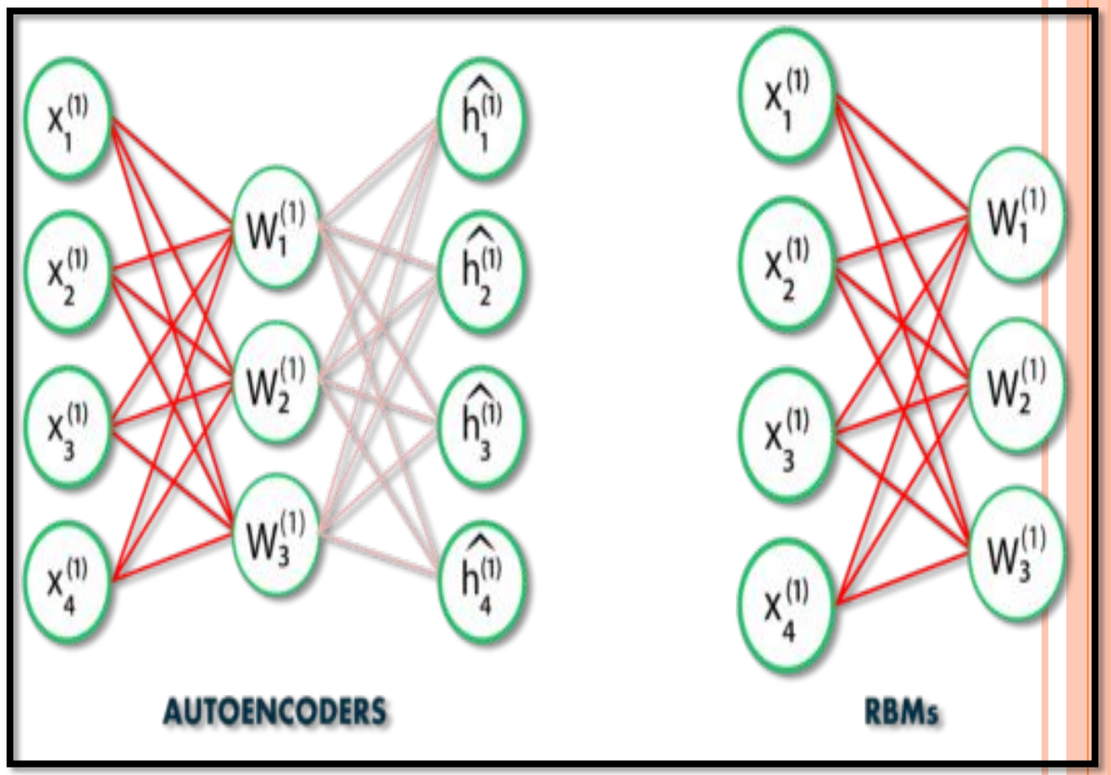
$$\mathcal{L}_{VAE} = \underbrace{\text{Reconstruction Loss}(x, \hat{x})}_{\text{Data fidelity}} + \underbrace{D_{KL}(q(z|x)\|p(z))}_{\text{Latent regularization}}$$

Mathematically,

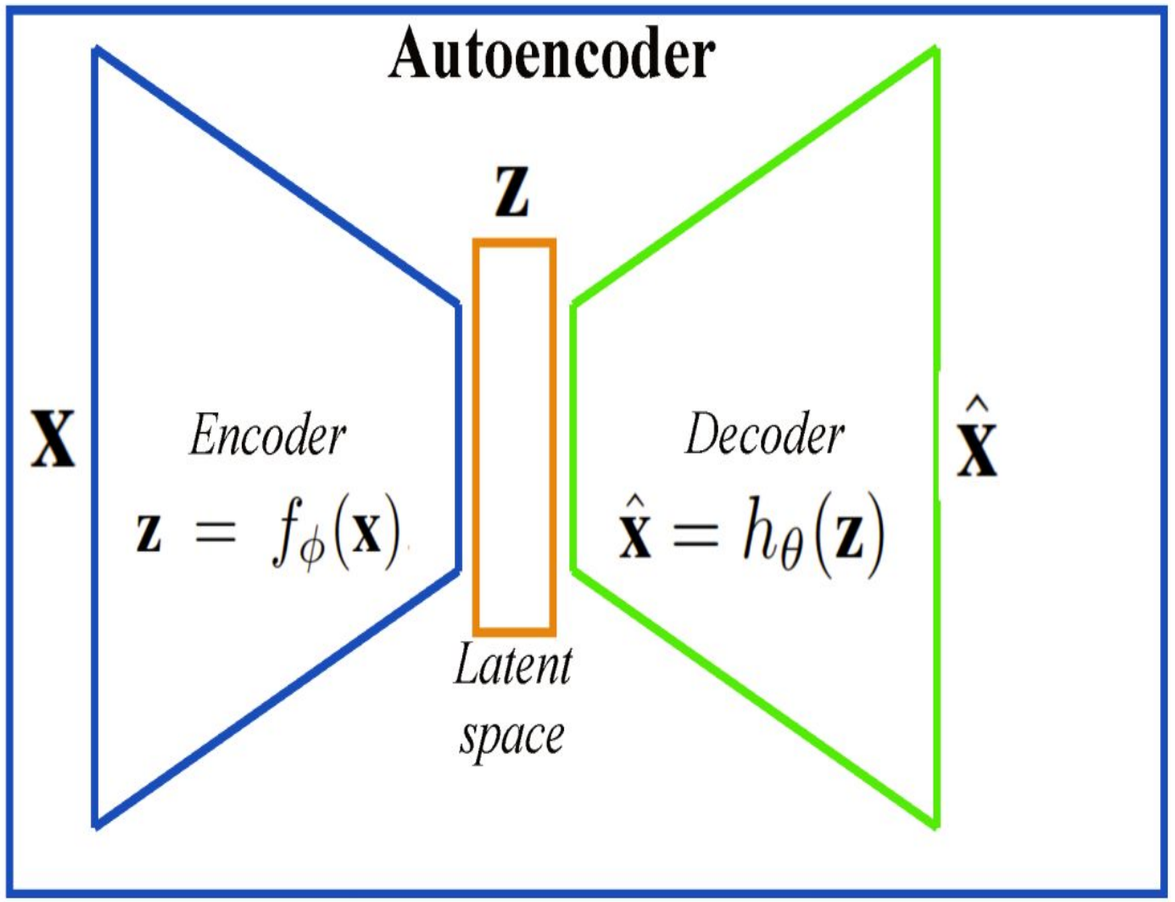


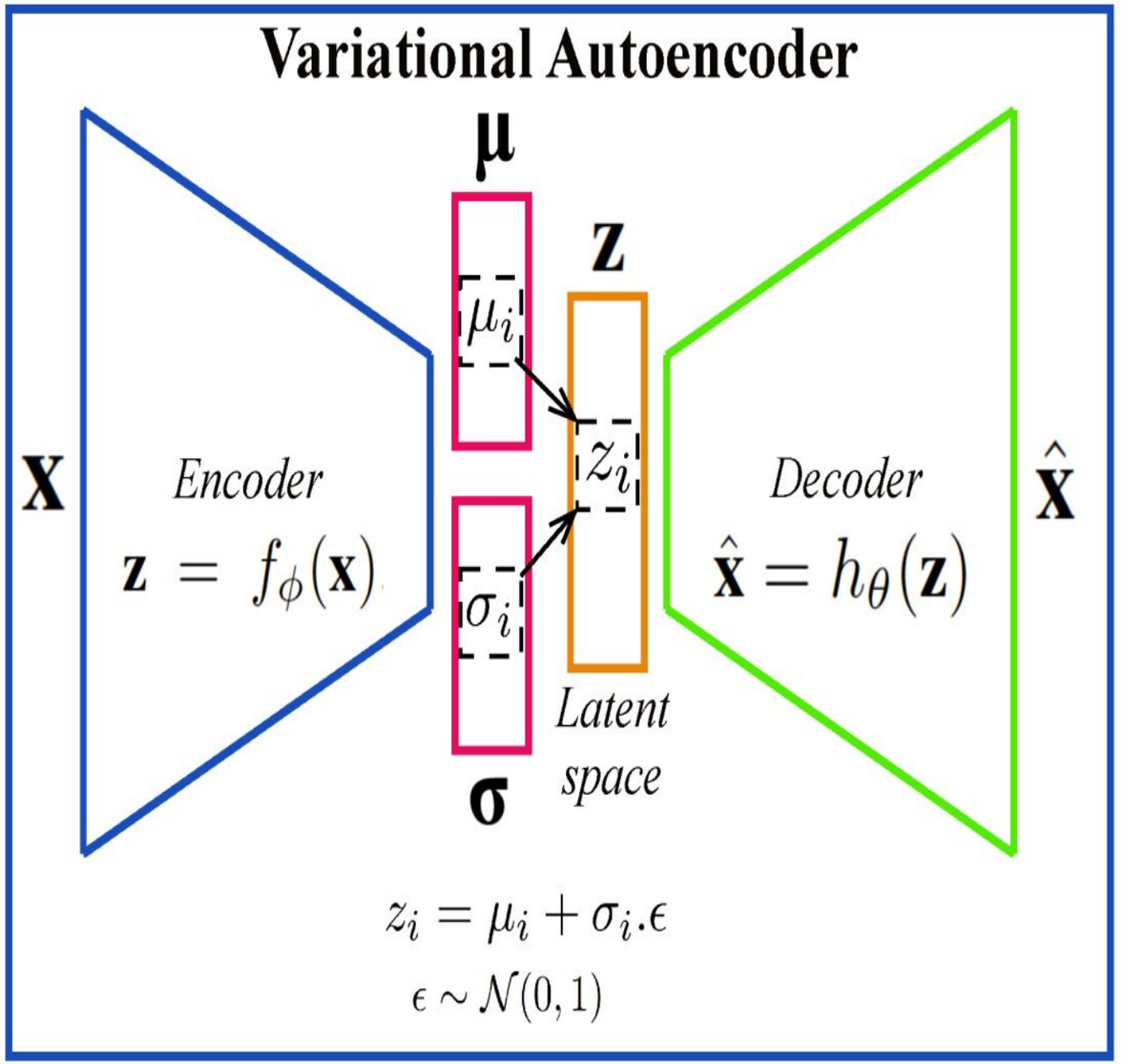
Difference between RBN and Autoencoder

- **Autoencoder** is a **simple 3-layer neural network** where **output units are directly connected back to input units.** Typically, the **number of hidden units** is **much less than the number of visible ones.**
- The **task of training** is to **minimize an error or reconstruction**, i.e. find the most efficient compact representation for input data.
- **Restricted Boltzmann Machines** are, **two-layer neural nets** that constitute the **building blocks of deep-belief networks**. The first layer of the RBM is called the **visible, or input layer, and the second is the hidden layer.**

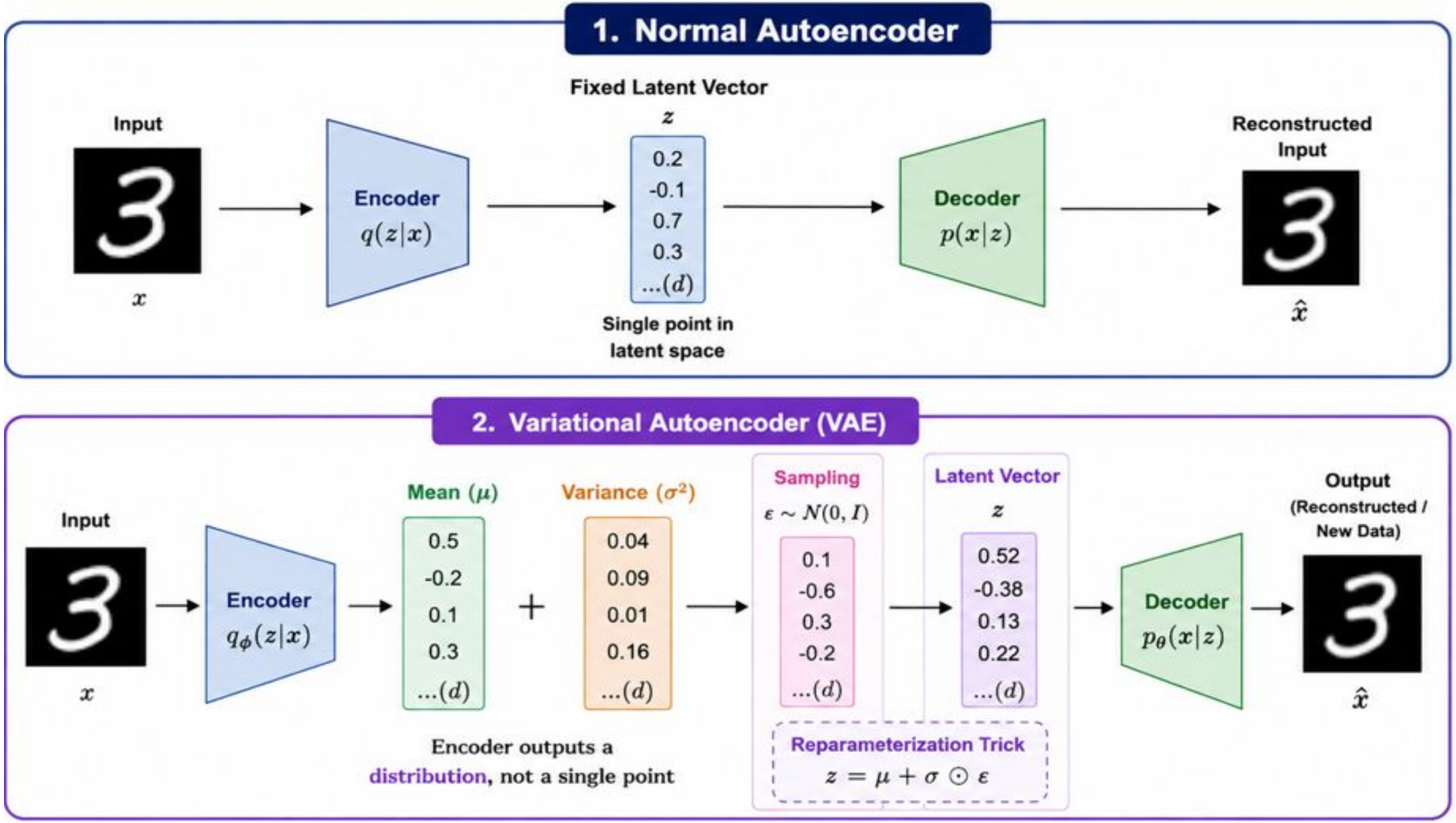


Difference b/w Autoencoder and Variational Autoencoder





Difference b/w Autoencoder and Variational Autoencoder



Autoencoders and Variational Autoencoders

Similarities & Differences

Similarities	Differences
Both AE& VAE are **Neural Network Architectures** that are used for **Unsupervised Learning**	AE uses **Deterministic Mapping** to represent the Input Data, while **VAE uses Probabilistic Mapping**
Both AE& VAE consists of an **encoder and decoder network.** The Encoder maps the input data to a latent representation, and the decoder maps the latent representation back to the original data	VAE has **additional Loss term** that is used to regularize the **latent space called KL Divergence Loss.** This loss uses the **Gaussian Distribution.**
Both AE and VAE are used for **Dimensionality Reduction, Data Generation and Anomaly Detection**	VAE can be used to **generate new data samples by sampling** from the **latent space, where AE cannot.**

Difference between Autoencoder & VAE

Autoencoder	Variational Autoencoder
Used to **generate a compressed transformation** of **input in a latent space**	Enforces conditions on the latent variable to be the **unit norm**
The latent variable is **not regularized**	The latent variable in the **compressed form** is **mean and variance**
Picking a random latent variable will generate **garbage output**	A random value of latent variable generates **meaningful output at the decoder**
Latent variable has **deterministic values**	The input of the decoder is **stochastic and is sampled from a Gaussian with mean and variance** of the output of the encoder.
The **latent space lacks the generative capability**	The **latent space has generative capabilities.**

VARIATIONAL AUTOENCODER

Thank You!

For Your Attention
and Valuable Time

Learning • Generating • Understanding
The Power of Probabilistic Latent Representations

